

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include <algorithm>

using namespace std;
using namespace chrono;

//----- MAX HEAP FUNCTIONS -----//

void createMaxHeap(vector<int> &data, int size, int root)
{
    int largest = root;
    int leftChild = 2 * root + 1;
    int rightChild = 2 * root + 2;

    if (leftChild < size && data[leftChild] > data[largest])
        largest = leftChild;

    if (rightChild < size && data[rightChild] > data[largest])
        largest = rightChild;

    if (largest != root)
    {
        swap(data[root], data[largest]);
        createMaxHeap(data, size, largest);
    }
}
```

```
}
```

```
void performMaxHeapSort(vector<int> &data)
```

```
{
```

```
    int size = data.size();
```

```
    for (int i = size / 2 - 1; i >= 0; i--)
```

```
        createMaxHeap(data, size, i);
```

```
    for (int i = size - 1; i > 0; i--)
```

```
    {
```

```
        swap(data[0], data[i]);
```

```
        createMaxHeap(data, i, 0);
```

```
    }
```

```
}
```

```
//----- MIN HEAP FUNCTIONS -----//
```

```
void createMinHeap(vector<int> &data, int size, int root)
```

```
{
```

```
    int smallest = root;
```

```
    int leftChild = 2 * root + 1;
```

```
    int rightChild = 2 * root + 2;
```

```
    if (leftChild < size && data[leftChild] < data[smallest])
```

```
        smallest = leftChild;
```

```
    if (rightChild < size && data[rightChild] < data[smallest])
```

```
        smallest = rightChild;
```

```

    if (smallest != root)
    {
        swap(data[root], data[smallest]);
        createMinHeap(data, size, smallest);
    }
}

void performMinHeapSort(vector<int> &data)
{
    int size = data.size();

    for (int i = size / 2 - 1; i >= 0; i--)
        createMinHeap(data, size, i);

    for (int i = size - 1; i > 0; i--)
    {
        swap(data[0], data[i]);
        createMinHeap(data, i, 0);
    }

    reverse(data.begin(), data.end());
}

//----- MAIN FUNCTION -----//

int main()
{
    int totalElements;

```

```
cout << "Enter the number of elements: ";
cin >> totalElements;

vector<int> numbers(totalElements);

srand((unsigned)time(NULL));

for (int i = 0; i < totalElements; i++)
{
    numbers[i] = rand() % 100000;
}

vector<int> maxHeapNumbers = numbers;
vector<int> minHeapNumbers = numbers;

// Max Heap Sort Timing
auto maxStart = high_resolution_clock::now();
performMaxHeapSort(maxHeapNumbers);
auto maxFinish = high_resolution_clock::now();

// Min Heap Sort Timing
auto minStart = high_resolution_clock::now();
performMinHeapSort(minHeapNumbers);
auto minFinish = high_resolution_clock::now();

auto maxNano = duration_cast<nanoseconds>(maxFinish - maxStart);
auto maxMicro = duration_cast<microseconds>(maxFinish - maxStart);
```

```

auto minNano = duration_cast<nanoseconds>(minFinish - minStart);
auto minMicro = duration_cast<microseconds>(minFinish - minStart);

cout << "\n===== MAX HEAP SORT =====\n";
cout << "Execution Time (Nanoseconds) : " << maxNano.count() << " ns" << endl;
cout << "Execution Time (Microseconds) : " << maxMicro.count() << " us" << endl;

cout << "\n===== MIN HEAP SORT =====\n";
cout << "Execution Time (Nanoseconds) : " << minNano.count() << " ns" << endl;
cout << "Execution Time (Microseconds) : " << minMicro.count() << " us" << endl;

return 0;
}

```

OUTPUT:

Enter the number of elements: 20

===== MAX HEAP SORT =====

Execution Time (Nanoseconds) : 2991 ns

Execution Time (Microseconds) : 2 us

===== MIN HEAP SORT =====

Execution Time (Nanoseconds) : 3320 ns

Execution Time (Microseconds) : 3 us

=== Code Execution Successful ===